Reconstructing a Multi-Stage Cyberattack with a Relational Database: A 3NF Schema and SQL Analytics over the AIT-LDSv2.0 Log Corpus

Naman Sudan, River Roseveare-Hunt, Ishaan Shetty
Department of Applied Data Science
San Jose State University
DATA 201, Spring 2026 (Group 5)
{naman.sudan, river.roseveare-hunt, ishaan.shetty}@sjsu.edu

Abstract*: We design, load, and exercise a third normal form (3NF) relational database over the AIT Log Data Set V2.0 (AIT-LDSv2.0), a publicly released enterprise security log corpus that captures a simulated multi-stage cyberattack across 22 networked hosts. The `russellmitchell` testbed slice contains 295,243 raw log lines across eight sources and 61,862 ground-truth attack labels spanning seven kill-chain phases. Our schema decomposes a heterogeneous mix of YAML, key-value `auditd`, and JSONL data into 22 normalized tables organized into three domains (host inventory, audit events, attack labels), enforced by primary and foreign key constraints in PostgreSQL 16. We articulate the normalization journey from staging tables (which preserve source-faithful 1NF violations) to 3NF using surrogate keys, lookup tables, and supertype/subtype decomposition of `auditd` records. On top of the loaded database we present a portfolio of 23 SQL query items (7 basic and 16 advanced, committed across 6 basic and 13 advanced `.sql` files) that cover subqueries, common table expressions, window functions, multi-table joins of three or more tables, performance indexing with `EXPLAIN ANALYZE`, and a saved view. The portfolio answers concrete attack-investigation questions: from baseline service activity, to detection-rule effectiveness, to a CTE-and-window-function reconstruction of the privilege-escalation sequence on a single host that joins seven tables. We further demonstrate an indexing improvement on the incident-response access pattern (composite index `idx_audit_event_host_timestamp`, execution time 1.462 ms to 0.640 ms, buffers 141 to 33), a saved view `v_privilege_escalation_timeline` that collapses an eight-table cross-domain join into a single virtual relation, and an interactive Streamlit dashboard exposing nine visualizations (one filter-driven KPI panel and eight story panels) directly bound to these queries. Our findings show that exfiltration-related labels dominate the corpus by line count (53,000+ lines across two log sources), while privilege escalation, the most operationally critical phase, leaves only 82 labeled lines but spans five distinct log sources, illustrating why a relational schema with cross-source joins is more diagnostic than any single log file in isolation.

I. Introduction

A. Motivation

Modern enterprise networks emit log streams from a heterogeneous mix of systems: web servers, audit daemons, virtual private network endpoints, name-resolution services, and authentication subsystems. Security operations teams must reason across these streams to detect attacker activity, attribute events to a kill-chain phase, and reconstruct a timeline after the fact. In practice, the work is hampered because each log type lives in its own format, storage location, and tool. A relational database offers a single substrate where structured queries can join evidence across sources and ground-truth labels.

This project takes a published, labeled, multi-host security log corpus [1], [2] and asks whether a single PostgreSQL database in third normal form can support a meaningful cross-log analysis of a real multi-stage attack. The dataset ships with attacker ground truth: every log line that participates in the simulated attack chain is labeled with one or more attack-phase identifiers, allowing us to quantify both detection coverage and the cost of false positives.

B. Problem and Research Gap

The AIT-LDSv2.0 corpus is large and heterogeneous: 295,243 log lines from eight sources on the `russellmitchell` testbed, with 61,862 labeled attack events across seven kill-chain phases. Most published exploration of the dataset treats each log file independently, or flattens the entire corpus into a single denormalized table, losing the relational structure that makes cross-source queries cheap. We close that gap by:

- Designing a 3NF schema that explicitly separates the host inventory, audit event, and attack-label domains while preserving cross-domain provenance and foreign keys.
- Implementing a reproducible ETL from raw files through staging tables into the final 3NF schema, with Alembic-managed migrations.
- Demonstrating that the resulting database supports analytical SQL expressive enough to reconstruct attack sequences and to compare detection-rule effectiveness across log sources.

C. Scope and Story Arc

Across the eight log sources in the `russellmitchell` slice, the attack chain traces a coherent sequence: a web foothold on `intranet_server`'s `access.log.2` during the `initial_access` phase, a `su` then `sudo cat /etc/shadow` chain captured by the host's `audit.log` during the `privilege_escalation` phase, and a DNS-tunneling `put` service on `internal_share` during the `exfiltration` phase roughly nine hours later. The narrative through this report follows the lead-in plus climax pair on `intranet_server`; the exfiltration phase is reported as "what happened next" in Section VII and reproduced in the appendix.

D. Project Goals

The project has four explicit deliverables, each of which is reflected in a section of this report:

1) A normalized relational schema with documented PK/FK and normalization decisions (Section III).
2) A reproducible PostgreSQL load pipeline with row-count verification (Section IV).
3) A portfolio of 23 SQL query items (7 basic and 16 advanced, spread across 6 basic and 13 advanced `.sql` files) that answer concrete attack-investigation questions, including indexing/`EXPLAIN` performance work and a saved view (Section V).
4) An interactive dashboard with nine visualizations that surfaces the query results to a non-SQL audience (Section VI).

II. Dataset and Data Processing

A. Source

We use the *russellmitchell* testbed slice of the AIT Log Data Set V2.0 (AIT-LDSv2.0) [1], [2]. The corpus simulates a four-day attack window (January 21 to 25, 2022, UTC) on a 22-host enterprise network running a mix of Ubuntu and Debian distributions. The dataset is publicly available under a permissive research license at the URL given in the bibliography entry.

B. Scale and Heterogeneity

The *russellmitchell* slice contains 295,243 raw log lines across eight source files on five hosts and 61,862 ground-truth attack labels spanning seven kill-chain phases (reconnaissance, initial access, privilege escalation, lateral movement, exfiltration, command and control, persistence). Forty-six unique attributes appear across log types. The corpus deliberately mixes formats to mirror real environments:

- **YAML**: `processing/config/servers.yaml` encodes the host inventory with multi-valued fields (`groups`, `fqdns`, `ipv4_addresses`, `ipv6_addresses`).
- **Linux `auditd`** key-value records: 3,048 events across two hosts (`intranet_server` and `internal_share`), capturing PAM authentication, service lifecycle, login, and command-execution traces.
- **JSONL labels**: 8 files containing per-line label annotations across 5 hosts, totalling 61,862 labeled events with 22 distinct label types.
- **Apache combined-log**, **OpenVPN syslog**, **dnsmasq**, **system auth** formats are present in the corpus but out of scope for the iteration documented here.

C. Why This Dataset

Two properties make AIT-LDSv2.0 a strong fit for a relational-database project. First, the corpus is *inherently denormalized*: multi-valued fields, key-value blobs, and overlapping record types force a real normalization exercise rather than a perfunctory 1NF pass. Second, the ground-truth labels enable cross-source queries with verifiable answers: "which detection rules fire on the privilege escalation phase" is a question with a numeric answer the database can compute, not a subjective judgment call.

D. Data Quality and Cleaning

The raw corpus has three classes of cleaning concerns that we document and address explicitly during ETL (Section IV):

1) **Multi-valued fields** (1NF violations) in the host YAML and in the `auditd msg` blob. We preserve them in staging tables (source-faithful) and decompose them into child tables during the staging-to-3NF transformation.
2) **Sentinel values**: `auid`/`ses`=4294967295 represent "unset." We preserve the raw value but document its semantics so query authors do not treat it as a real session.
3) **Sparse columns from heterogeneous record types**: a single `auditd` log file contains ≥ 8 structurally different record types (LOGIN, SYSCALL, AVC, PROCTITLE, PAM events, etc.). A flat-table representation would have $\sim 80\%$ NULL columns. We apply supertype/subtype decomposition (Section III) to push subtype-specific attributes out of the supertype.

E. Coverage Summary

Table I groups the eight `russellmitchell` source files by host and gives the labeled-line count and the dominant attack phase recorded against each one. The table makes the foothold-to-escalate story on `intranet_server` visible at a glance, and reminds the reader that the exfiltration evidence on `internal_share` also belongs to the same attack chain even though it lives on a different host.

III. Schema and Normalization

The relational design has three domains and 22 tables. The ER/EER diagram in Figure 1 shows the full schema; primary keys are underlined and foreign keys are rendered as directed edges. The normalization journey is documented in detail in `docs/schema/normalization_raw_to_3nf.md` and the relation catalog in `docs/schema/data_model_3nf.md`; this section summarizes the design rationale.

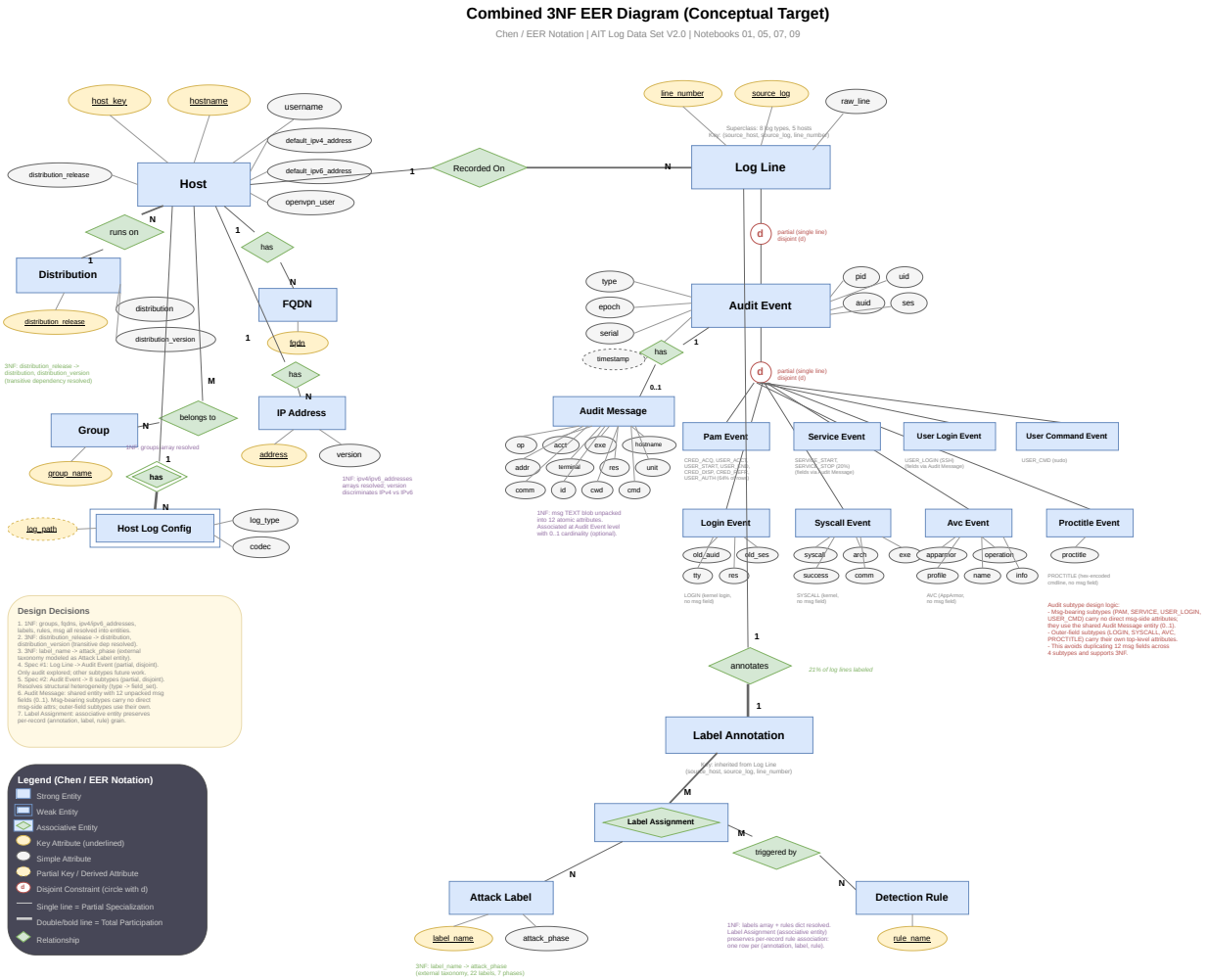

Fig. 1. Final 3NF EER schema: 22 tables across host inventory (7), audit events (10), and attack labels (5) domains. Surrogate primary keys (SERIAL) are used throughout to keep foreign-key columns narrow across the join graph.

TABLE I
LABELED-LINE COVERAGE BY SOURCE FILE. THE FIRST SIX ROWS ARE THE AUDIT AND HOST-SIDE EVIDENCE; THE FINAL ROW IS THE BULK DNS-TUNNELING CHANNEL THAT DRIVES THE CORPUS-WIDE LINE COUNT.

Host	Source log	Lines	Dominant phase
intranet_server	access.log.2	7,691	initial_access
intranet_server	auth.log	8	privilege_escalation
intranet_server	audit.log	9	privilege_escalation
internal_share	audit.log	2	exfiltration
inet-firewall	dnsmasq.log	12	privilege_escalation
monitoring	cpu.log	49	privilege_escalation
internal_share	dns.log	≈53,000	exfiltration

A. Domain 1: Host Inventory (7 tables)

The host domain captures the testbed inventory. The core entity is `host`, with row grain "one row per machine" and 22 rows total. Three multi-valued attributes (`groups`, `fqdns`, `ipv4_addresses`, `ipv6_addresses`) become 1NF child tables (`host_group`, `host_fqdn`, `host_ipv4`, `host_ipv6`) with composite primary keys. The transitive dependency `distribution_release` → {`distribution`, `distribution_version`} is removed by the 3NF lookup table `os_release` (2 rows: bionic and stretch). The `logs` composite multi-valued field becomes `host_log_config` (66 rows, 1 to 9 configs per host).

B. Domain 2: Audit Events (10 tables)

The audit domain demonstrates supertype/subtype decomposition. The supertype `audit_event` (3,048 rows) carries the attributes common to every `auditd` record (`event_id`, `host_id`, `type`, `epoch`, `timestamp`, `pid`, `uid`, `auid`, `ses`, `line_number`, `raw_line`). Two classes of subtypes extend it:

- **Msg-bearing subtypes**: `audit_pam_event`, `audit_service_event`, `audit_user_login_event`,

audit_user_cmd_event. The semantic content for these subtypes lives inside the msg=' ... ' key-value blob in the raw line. We resolve this 1NF violation by parsing the blob into 12 atomic columns in audit_message (2,614 rows), which holds a 0..1 relationship with audit_event.

- **Outer-field subtypes**: audit_login_event, audit_syscall_event, audit_avc_event, audit_proctitle_event. These record types carry their semantic content as top-level fields rather than inside msg; the subtype tables hold those fields directly.

The decomposition turns a hypothetical flat "audit_line" table with $\sim$ 44 columns and 80% NULL density into a supertype + 9 sparse subtype tables with full population per subtype. The cost is more joins in queries; the benefit is correct cardinality, query-planner statistics, and a cleaner mental model.

C. Domain 3: Attack Labels (5 tables)

The label domain links log lines to attacker activity. The grain is one row per *labeled* log line, identified by the natural key (source_host, source_log, line_number). labeled_line (61,862 rows) is the line-level fact table; labeled_line_label is the M:N junction connecting lines to labels; labeled_line_rule captures which detection rule generated each label (an event can be labeled by multiple rules). The taxonomy is normalized into two lookup tables: attack_label (22 rows: distinct label names) and attack_phase (7 rows: kill-chain phases), with a 1:N relationship from phase to label.

D. Normalization Decisions Worth Calling Out

Three decisions are explicit project rules rather than mechanical applications of the normalization algorithm:

1) **Surrogate keys throughout.** Every entity has a SERIAL primary key, even when the natural key (e.g. host_key, (source_host, source_log, line_number)) is unique. This keeps foreign-key columns narrow (INT rather than VARCHAR(50)) and decouples domain-key changes from join indexes. The natural key is preserved as a UNIQUE constraint.
2) **Provenance preserved on the supertype.** audit_event carries the original raw_line TEXT even after the parsed columns are populated. This is a deliberate space cost in exchange for reproducibility: we can re-derive any parsed field from the raw text if a downstream bug is found.
3) **One pragmatic 1NF exception.** host_log_config has an add_field_json TEXT column that retains the JSON payload of optional log-specific fields. Rather than create a polymorphic child table per log type (high schema cost, low query value), we leave the JSON opaque and query it with PostgreSQL jsonb_path_query when needed. The exception is documented and the column is unused by the queries in Section V.

E. 3NF Claim

For each non-key attribute in every relation, we verified that the attribute depends only on the primary key (no partial dependencies, as all primary keys are atomic SERIAL surrogates) and that it does not transitively depend on another non-key attribute. The two textbook candidates for transitive dependency in the source data (distribution_release $\rightarrow$ distribution, distribution_version) and the lookup-style relationship (label_name $\rightarrow$ phase_name) are removed by promoting them into separate lookup tables (os_release and attack_phase, respectively). The one documented exception is host_log_config.add_field_json, which is a TEXT blob carrying log-type-specific configuration values. We treat the column as opaque (queried only with PostgreSQL JSON functions when needed) rather than synthesizing a polymorphic child table. The exception is documented in docs/schema/data_model_3nf.md and the column is unused by any query in Section V.

IV. Database Setup and Loading

A. Stack

We use PostgreSQL 16 [3] running inside a single Docker container defined in docker-compose.yml [4]. Schema migrations are managed by Alembic [5] so that any team member can rebuild the database from a clean state with a single command. Application code is Python 3.12 with SQLAlchemy ORM models for parsing and loading.

B. Repository Layout

```
data-201-security-log-analysis/
|-- alembic/              schema migrations
|-- docker-compose.yml    Postgres 16
|-- src/
|   |-- models/           SQLAlchemy models
|   |-- parsers/          raw -> dict parsers
|   |-- loaders/          dict -> row loaders
|   `-- dashboard/        Streamlit app + db.py
|-- sql/
|   |-- 3nf/              per-table CREATE TABLE
|   |-- 3nf/_indexes.sql  canonical index DDL
|   |-- 3nf/_views.sql    canonical view DDL
|   |-- queries/basic/    6 basic .sql files
|   `-- queries/advanced/ 13 advanced .sql files
|-- notebooks/            10 exploration nbs
|-- docs/                 schema, ERD, findings
`-- report/               this report
```

C. Reproducible Load

The full load runs in five commands from a clean clone of the repository:

```
docker compose up -d                    #
    Postgres on :5432
python -m venv venv && source
    venv/bin/activate
pip install -r requirements.txt
alembic -c alembic/alembic.ini upgrade head
python -m src.loaders.run_all
```

The migration chain creates 22 tables across staging and 3NF schemas; the loader script reads the raw `russellmitchell` files, runs the parsers in `src/parsers/`, and inserts rows in dependency order (host inventory first, then audit events, then attack labels). The loader is idempotent: re-running against an already-loaded database truncates and re-inserts each table inside a transaction, so the output is reproducible regardless of starting state.

D. Migration Head and Schema Objects

After `alembic upgrade head` the database carries revision `742e860d116f`, which adds two analytical objects on top of the 22-table 3NF base:

- Composite index `idx_audit_event_host_timestamp` on `audit_event(host_id, timestamp)`, the index that powers the incident-response speedup documented in Section V-E.
- Saved view `v_privilege_escalation_timeline`, an eight-table cross-domain join filtered to `attack_phase.phase_name = 'privilege_escalation'` and aggregated with `string_agg`, used by both the dashboard and the deep-dive query in Section V-F.

A teammate can verify that both objects are present with one psql command:

```
docker exec security-logs-dev psql -U
    security_logs_user \
  -d security_logs \
  -c "SELECT version_num FROM
      alembic_version;" \
  -c "SELECT indexname FROM pg_indexes
      WHERE tablename = 'audit_event'
      ORDER BY indexname;" \
  -c "SELECT viewname FROM pg_views
      WHERE schemaname = 'public';"
```

Expected output: `version_num = 742e860d116f`, `idx_audit_event_host_timestamp` appears in the index list, and `v_privilege_escalation_timeline` appears in the view list.

E. Final Row Counts

Table II reports the final loaded row counts (verified 2026-05-01 against the loaded database). Total across all 22 tables is roughly 439,998 rows. The largest single table is `labeled_line_rule` (184,651 rows: a per-rule annotation on labeled lines), closely followed by `labeled_line_label` (184,517 rows: the line-to-label many-to-many junction).

F. Integrity Verification

After load we run a small set of integrity checks. The two most load-bearing checks are the orphan-FK count on `audit_event` and the cross-domain consistency check between `labeled_line` and `audit_event` for rows where the labeled log is `audit.log`. Both return zero on the current database.

TABLE II
FINAL LOADED ROW COUNTS BY DOMAIN (VERIFIED 2026-05-01).

Domain	Table	Rows
Host inventory	host	22
	host_group	63
	host_fqdn	20
	host_ipv4	24
	host_ipv6	24
	host_log_config	66
	os_release	2
Audit events	audit_event	3,048
	audit_message	2,614
	audit_pam_event	2,055
	audit_service_event	555
	audit_login_event	410
	audit_user_login_event	3
	audit_user_cmd_event	1
	audit_syscall_event	8
	audit_avc_event	8
	audit_proctitle_event	8
Attack labels	labeled_line	61,862
	labeled_line_label	184,517
	labeled_line_rule	184,651
	attack_label	22
	attack_phase	7
Total		**≈439,998**

```
-- Every audit_event references a real host (FK
    consistency).
SELECT COUNT(*) AS orphans
FROM audit_event ae
LEFT JOIN host h ON h.host_id = ae.host_id
WHERE h.host_id IS NULL;
-- => 0 orphans

-- Every labeled_line line_number that targets audit.log
    lands
-- inside its host's audit_event range.
SELECT COUNT(*) AS unmatched
FROM labeled_line ll
WHERE ll.source_log = 'audit.log'
  AND NOT EXISTS (
    SELECT 1 FROM audit_event ae
    JOIN host h ON h.host_id = ae.host_id
    WHERE h.host_key    = ll.source_host
      AND ae.line_number = ll.line_number
  );
-- => 0 unmatched

-- Subtype coverage: every audit_event has exactly one
    subtype row.
WITH subtype_sum AS (
  SELECT
    (SELECT COUNT(*) FROM audit_pam_event)
  + (SELECT COUNT(*) FROM audit_service_event)
  + (SELECT COUNT(*) FROM audit_user_login_event)
  + (SELECT COUNT(*) FROM audit_user_cmd_event)
  + (SELECT COUNT(*) FROM audit_login_event)
  + (SELECT COUNT(*) FROM audit_syscall_event)
  + (SELECT COUNT(*) FROM audit_avc_event)
  + (SELECT COUNT(*) FROM audit_proctitle_event)
    AS subtype_total
)
SELECT (SELECT COUNT(*) FROM audit_event) AS total_events,
       subtype_total AS sum_of_subtypes
FROM   subtype_sum;
-- => total_events = sum_of_subtypes = 3,048
```

The third check (subtype-coverage equality) is the practical proof that the supertype/subtype decomposition described in Section III is total and disjoint, which is the property that lets

the report claim "every audit event is exactly one subtype."

V. SQL Analysis and Results

A. Methodology

The query portfolio contains **23 SQL query items**: 7 basic and 16 advanced, committed across **6 basic and 13 advanced** `.sql` files in the project repository under `sql/queries/basic/` and `sql/queries/advanced/`. (Two files each hold more than one query: `naman_basic_queries.sql` holds two basic queries B6 and B7, and `naman_advanced_queries.sql` holds four advanced queries A4 through A7.) The portfolio is augmented by one saved view (`v_privilege_escalation_timeline`) and one composite index (`idx_audit_event_host_timestamp`), both managed by Alembic migration `742e860d116f`. Together they cover all six SQL categories called out by the rubric: subqueries, common table expressions, window functions, multi-table joins of three or more tables, performance indexing with `EXPLAIN ANALYZE`, and saved views.

Every query is reproducible. A reader who has followed the load steps in Section IV can paste any `SELECT` below into their psql session and obtain the same numeric result. For queries where the underlying file in the repository contains more text (header comments documenting story role, expected result, or recommendation), the report includes the executable `SELECT` body and refers the reader to the file path for the full annotation.

For the longer deep-dive queries (A4, A5, A6, A7, the EXPLAIN demonstration, and the saved view) we include the question, the SQL listing, a numeric result summary, and an interpretation paragraph. For the simpler queries we keep the same four elements but compress the SQL into a smaller listing block.

B. Portfolio Matrix

Table III lists every SQL query in the portfolio with its owner, classification, technique tag, story role, and source file. The matrix is the canonical mapping for the rubric: every query the professor expects to count is on this table.

C. Basic Queries

1) B1 (River): Labeled lines per source host: **Question.** Which hosts contribute the most labeled attack evidence across the corpus?

```
SELECT source_host,
       COUNT(*) AS labeled_line_count
FROM labeled_line
GROUP BY source_host
ORDER BY labeled_line_count DESC;
```

Result. Five rows, one per host that carries at least one labeled line. `internal_share` dominates by raw count because of the DNS-tunneling exfiltration; `intranet_server` contributes the bulk of the foothold lines from `access.log.2`; and `inet-firewall` plus `monitoring` contribute the low-volume side-channel labels.

Insight. The dominant host frames the rest of the analysis. Once the team saw that `intranet_server` is the only host with labels distributed across multiple phases (`initial_access`, `privilege_escalation`, plus side-channel `exfiltration`-adjacent labels), it became the natural focal host for the deep-dive queries.

2) B2 (River): Attack labels per phase: **Question.** How rich is the label taxonomy in each attack phase?

```
SELECT ap.phase_name,
       COUNT(al.label_id) AS label_count
FROM attack_phase ap
FULL OUTER JOIN attack_label al
  ON al.phase_id = ap.phase_id
GROUP BY ap.phase_id, ap.phase_name
ORDER BY label_count DESC;
```

Result. Seven rows (one per phase) showing label counts in the range one to seven. `exfiltration` and `privilege_escalation` carry the most distinct labels; `persistence` and `lateral_movement` are sparsest.

Insight. Phases with richer taxonomies tend to be the phases the dataset authors instrumented most carefully. The label-count distribution shows where the dataset's designers expected detection sophistication to land.

3) B3 (River): Audit-event count per host: **Question.** Which hosts produce `auditd` traces at all, and how many events does each one emit?

```
SELECT h.hostname,
       h.host_key,
       COUNT(ae.event_id) AS event_count
FROM host h
INNER JOIN audit_event ae
  ON ae.host_id = h.host_id
GROUP BY h.host_id, h.hostname, h.host_key
ORDER BY event_count DESC;
```

Result. Two rows: `intranet_server` (2,316 events) and `internal_share` (732 events).

Insight. Only two of the 22 testbed machines emit `auditd`, which is consistent with the testbed configuration in `servers.yaml`. This is the sole reason the audit domain has 3,048 rows in total: there is no missing data, the corpus simply constrains audit instrumentation to two hosts.

4) B4 (Ishaan): Labeled lines per attack phase: **Question.** How heavily is each phase represented in the labeled data, by line count?

```
SELECT ap.phase_name, COUNT(*) AS labeled_lines
FROM attack_phase ap
JOIN attack_label al        ON al.phase_id  =
    ap.phase_id
JOIN labeled_line_label lll ON lll.label_id =
    al.label_id
GROUP BY ap.phase_name
ORDER BY labeled_lines DESC;
```

Result. Seven rows. `exfiltration` dominates with roughly 53,000 lines (driven by per-DNS-query labels on `internal_share/dns.log`); `initial_access` carries the second-largest count (driven by foothold

TABLE III

FULL SQL PORTFOLIO MATRIX (MID-PRESENTATION PLUS FINAL-PRESENTATION QUERIES). FILE PATHS IN THE RIGHTMOST COLUMN ARE RELATIVE TO SQL/QUERIES/<CLASS>/ WHERE <CLASS> IS BASIC OR ADVANCED PER THE THIRD COLUMN.

ID	Owner	Class	Technique	Story role	File
B1	River	basic	SELECT, GROUP BY, ORDER BY	breadth (labeled lines per host)	river_lines_per_host.sql
B2	River	basic	FULL OUTER JOIN, GROUP BY	taxonomy (labels per phase)	river_attack_labels_per_host.sql
B3	River	basic	INNER JOIN, GROUP BY	breadth (audit volume per host)	river_audit_labels_per_host.sql
B4	Ishaan	basic	3-table JOIN, GROUP BY	taxonomy (lines per phase)	ishaan_labeled_lines_per_attack_phase.sql
B5	Ishaan	basic	GROUP BY, LIMIT	detection volume (top-10 rules)	ishaan_top10_label_rules.sql
B6	Naman	basic	4-table JOIN, CASE inside SUM	baseline (service activity)	naman_basic_queries.sql (Q1)
B7	Naman	basic	4-table JOIN, COUNT DISTINCT	breadth (phase by source-log spread)	naman_basic_queries.sql (Q2)
A1	Ishaan	adv	EXISTS subquery	breadth (event types on labeled hosts)	ishaan_distinct_event_types_with_at_least1_labeled_log.sql
A2	River	adv	2 CTEs + CROSS JOIN	breadth (% of labeled lines per source)	river_labeled_lines_by_host_and_log.sql
A3	Ishaan	adv	RANK() OVER PARTITION	temporal (busiest day per host)	ishaan_events_per_day_and_busiest_ranked.sql
A4	Naman	adv	CTE + 7-table JOIN + string_agg	climax (priv-esc reconstruction)	naman_advanced_queries.sql (Q1)
A5	Naman	adv	CTE + DENSE_RANK window	detection ranking (all phases)	naman_advanced_queries.sql (Q2)
A6	Naman	adv	CTE + 7-table JOIN + COALESCE	lateral movement	naman_advanced_queries.sql (Q3)
A7	Naman	adv	CTE + LAG + ROW_NUMBER	burst detection (timing only)	naman_advanced_queries.sql (Q4)
A8	Ishaan	adv	EXISTS subquery (4-table body)	climax (audit types in priv-esc)	ishaan_audit_types_in_privilege_escalation.sql
A9	Ishaan	adv	RANK() over per-day counts	bridge (busiest day on intranet)	ishaan_intranet_server_busiest_day.sql
A10	Ishaan	adv	CTE + ROW_NUMBER OVER PARTITION	coverage (top rule per label)	ishaan_top_rule_per_label_priv_esc.sql
A11	River	adv	CTE + NTILE(3) window	lead-in (foothold-to-escalate)	river_foothold_to_escalate_journey.sql
A12	River	adv	CTE + cumulative SUM window	lead-in (foothold accumulation)	river_foothold_cumulative.sql
A13	River	adv	CTE + RANK() window	breadth (priv-esc source spread)	river_priv_esc_source_spread.sql
A14	Naman	adv	CTE + DENSE_RANK (priv-esc only)	detection ranking (priv-esc only)	naman_rule_effectiveness_priv_esc.sql
View	Naman	saved	8-table JOIN + string_agg, GROUP BY	climax (priv-esc timeline)	naman_view_privilege_escalation_timeline.sql DDL: sql/3nf/_views.sql
Index	Naman	perf	CREATE INDEX + EXPLAIN ANALYZE	incident response (host + window)	naman_explain_index_improvement.sql DDL: sql/3nf/_indexes.sql

labels on intranet_server/access.log.2); privilege_escalation carries only 82 lines.

Insight. Volume is a misleading detection metric on this dataset: the lowest-count phase (privilege escalation) is the most operationally significant. Section VII returns to this point.

5) B5 (Ishaan): Top-10 detection rules: **Question.** Which detection rules contribute the most label firings?

```
SELECT rule_name, COUNT(*) AS times_used
FROM labeled_line_rule
GROUP BY rule_name
ORDER BY times_used DESC
LIMIT 10;
```

Result. Ten rows. dnsteal.domain.match is rank one with roughly 53,000 firings, exfiltration-adjacent rules round out the top half, and audit-derived rules appear in the bottom half with low absolute counts but high per-firing severity.

Insight. The top-10 view by raw firings buries the audit-domain rules. Query A14 (Section V-D14) re-ranks the same data after restricting to the privilege_escalation phase, which surfaces those audit rules to the top.

6) B6 (Naman, Q1 of naman_basic_queries.sql): Service activity baseline: **Question.** What is the baseline service activity on the two audit-instrumented hosts? Establishing the baseline lets later queries see which one stop/start pair stands out as anomalous.

```
SELECT h.host_key,
       am.unit AS service_name,
       SUM(CASE WHEN ae.type = 'SERVICE_START'
                THEN 1 ELSE 0 END) AS starts,
       SUM(CASE WHEN ae.type = 'SERVICE_STOP'
                THEN 1 ELSE 0 END) AS stops
FROM audit_event ae
JOIN audit_service_event ase ON ase.event_id =
    ae.event_id
JOIN audit_message am        ON am.event_id  =
    ae.event_id
JOIN host h                  ON h.host_id    =
    ae.host_id
GROUP BY h.host_key, am.unit
ORDER BY h.host_key,
         SUM(CASE WHEN ae.type = 'SERVICE_START'
                  THEN 1 ELSE 0 END)
       + SUM(CASE WHEN ae.type = 'SERVICE_STOP'
                  THEN 1 ELSE 0 END) DESC;
```

Result. 33 (host, service) rows. phpsessionclean dominates intranet_server with 384 events. The put service on internal_share has a single start and a single stop amid hundreds of routine cron-driven service events.

Insight. The put service start/stop pair is the exfiltration service whose existence is later confirmed by attack labels. The query is basic in its SQL, but it is the rare basic query that produces an actionable detection signal on its own.

7) B7 (Naman, Q2 of naman_basic_queries.sql): Phase coverage by log-source spread: **Question.** Which attack phase generates evidence in the most distinct log sources, and what is the line count behind that spread?

```
SELECT ap.phase_name,
       COUNT(DISTINCT lll.labeled_line_id) AS
           labeled_lines,
       COUNT(DISTINCT ll.source_host || '/' ||
           ll.source_log)
         AS log_sources_affected
FROM attack_phase ap
JOIN attack_label al         ON al.phase_id  =
    ap.phase_id
JOIN labeled_line_label lll ON lll.label_id =
    al.label_id
JOIN labeled_line ll
  ON ll.labeled_line_id = lll.labeled_line_id
GROUP BY ap.phase_name
ORDER BY labeled_lines DESC;
```

Result. Seven rows. exfiltration dominates by line count (53K) but spans only two log sources; privilege_escalation carries 82 lines but spans five distinct (source_host, source_log) pairs (audit on host, auth, access, cpu, dnsmasq).

Insight. Spread and volume rank the phases differently. The spread metric is what motivates the cross-source joins in

queries A4 through A7 and the saved view: a phase that lives on five logs is one the database can describe more completely than any single log can.

D. Advanced Queries

1) A1 (Ishaan): Distinct event types on labeled hosts (EXISTS subquery): **Category.** Subquery (`EXISTS`).
Question. Which audit event types appear on hosts that have at least one labeled log line, anywhere in the corpus?

```
SELECT DISTINCT e.type
FROM audit_event e
WHERE EXISTS (
  SELECT 1
  FROM host h
  JOIN labeled_line ll
    ON ll.source_host = h.host_key
  WHERE h.host_id = e.host_id
);
```

Result. 15 distinct audit-event type values: PAM, service lifecycle, login, syscall, AVC, proctitle, user-cmd, user-login.

Insight. The `EXISTS` subquery is a simple existence test, but it confirms that every audit-event type observed in the corpus also coexists with at least one labeled line on the same host. There is no audit-event type that is exclusively benign in the recorded data, which justifies treating the audit subtypes as a unified analysis surface in queries A4 onwards.

2) A2 (River): Cross-source distribution of labeled lines (multi-CTE + CROSS JOIN): **Category.** Common table expression (two CTEs) + `CROSS JOIN`.
Question. What share of labeled lines comes from each `(source_host, source_log)` pair?

```
WITH per_source AS (
    SELECT source_host, source_log, COUNT(*) AS
        line_count
    FROM labeled_line
    GROUP BY source_host, source_log
),
total AS (
    SELECT SUM(line_count) AS total_count FROM
        per_source
)
SELECT ps.source_host,
       ps.source_log,
       ps.line_count,
       ROUND(100.0 * ps.line_count /
           t.total_count, 2)
         AS pct_of_all_labeled
FROM per_source ps
CROSS JOIN total t
ORDER BY ps.line_count DESC;
```

Result. Eight rows, one per labeled `(source_host, source_log)` pair. The top row is `internal_share/dns.log` with 53%+ of all labeled lines; `intranet_server/access.log.2` is next at roughly 12%; and a long tail covers the remaining six pairs at low single digits.

Insight. The `CROSS JOIN` against the single-row `total` CTE is the standard SQL idiom for a percentage column that does not require a window function. The result is the canonical "fat head" shape that explains why the abstract emphasises spread over volume: most of the corpus is one phase on one log.

3) A3 (Ishaan): Busiest day per host (window function): **Category.** Window function (`RANK() OVER PARTITION BY`).
Question. On what day did each host generate the most audit events?

```
SELECT h.host_key,
       DATE(e.timestamp) AS event_date,
       COUNT(*) AS events_that_day,
       RANK() OVER (
           PARTITION BY h.host_key
           ORDER BY COUNT(*) DESC
       ) AS day_rank_for_host
FROM audit_event e
JOIN host h ON h.host_id = e.host_id
GROUP BY h.host_key, DATE(e.timestamp)
ORDER BY h.host_key, day_rank_for_host, event_date;
```

Result. Eight rows: four event-days for `intranet_server` and four for `internal_share`, ranked within each host. On `intranet_server` the busiest day is 2022-01-23; the attack day (2022-01-24) ranks third.

Insight. The attacker's audit activity is below the host's baseline, not above it. This is the key fact behind the "hides in baseline" framing in Section VI: a detection rule that triggers only on per-day audit-event-count outliers would miss the attack on this host.

4) A4 (Naman, Q1 of `naman_advanced_queries.sql`): Privilege-escalation reconstruction: **Category.** CTE + `string_agg` over a grouping + multi-table join. The CTE body joins **seven** tables: `audit_event`, `host`, `labeled_line`, `labeled_line_label`, `attack_label`, `attack_phase`, and a `LEFT JOIN` to `audit_message` (so the PAM message columns are nullable on events that have no msg blob).
Question. Can we reconstruct the exact privilege-escalation attack sequence on `intranet_server`?

```
WITH attack_timeline AS (
    SELECT ae.event_id,
           ae.timestamp,
           ae.type AS audit_type,
           am.op   AS pam_operation,
           am.acct AS target_account,
           am.exe  AS executable,
           am.terminal,
           al.label_name,
           ap.phase_name
    FROM audit_event ae
    JOIN host h          ON h.host_id = ae.host_id
    JOIN labeled_line ll ON ll.source_host =
        h.host_key
                        AND ll.source_log  =
                            'audit.log'
                        AND ll.line_number =
                            ae.line_number
    JOIN labeled_line_label lll
      ON lll.labeled_line_id = ll.labeled_line_id
    JOIN attack_label al ON al.label_id =
        lll.label_id
    JOIN attack_phase ap ON ap.phase_id =
        al.phase_id
```

```
    LEFT JOIN audit_message am ON am.event_id =
        ae.event_id
    WHERE h.host_key = 'intranet_server'
)
SELECT event_id, timestamp, audit_type,
    pam_operation,
       target_account, executable, terminal,
       string_agg(label_name, ', ' ORDER BY
           label_name) AS labels
FROM attack_timeline
GROUP BY event_id, timestamp, audit_type,
    pam_operation,
        target_account, executable, terminal
ORDER BY timestamp, event_id;
```

Result. Nine rows that decompose into two distinct attack bursts:

- Burst 1 at 2022-01-24 04:37:40 UTC: a `su` from `www-data` to `jhall` via `/bin/su` (four PAM events: USER_AUTH, USER_ACCT, CRED_REFR, USER_START).
- Burst 2 at 2022-01-24 04:38:06 UTC: `sudo cat /etc/shadow` via `/usr/bin/sudo` as `jhall` (five events: USER_CMD plus four PAM events). Burst 2 begins 25.6 seconds after burst 1.

Insight. The reconstruction is only possible because audit events, host inventory, and attack labels live in the same database with cross-domain provenance keys. Each of the seven joined tables contributes an irreducible fact: `audit_event` for the timeline, `audit_message` for the executable name and target account, `host` for the human-readable host key, `labeled_line` for the audit-log line number, `labeled_line_label` and `attack_label` for the ground-truth label, and `attack_phase` for the kill-chain phase. The query is the rubric's "multi-table join (three or more tables)" line item with margin to spare.

5) A5 (Naman, Q2 of `naman_advanced_queries.sql`): Detection-rule effectiveness across all phases: **Category.** CTE + window function (`DENSE_RANK`) + three-table join. **Question.** Across every kill-chain phase, which detection rules are most effective at identifying labeled attacker activity?

```
WITH rule_stats AS (
    SELECT llr.rule_name,
           ap.phase_name,
           COUNT(DISTINCT llr.labeled_line_id)
             AS lines_triggered,
           COUNT(DISTINCT llr.label_id)
             AS labels_triggered
    FROM labeled_line_rule llr
    JOIN attack_label al ON al.label_id =
        llr.label_id
    JOIN attack_phase ap ON ap.phase_id =
        al.phase_id
    GROUP BY llr.rule_name, ap.phase_name
)
SELECT rule_name,
       phase_name,
       lines_triggered,
       labels_triggered,
       DENSE_RANK() OVER (ORDER BY lines_triggered
           DESC)
         AS effectiveness_rank
FROM rule_stats
ORDER BY lines_triggered DESC
LIMIT 15;
```

Result. 15 rows. `dnsteal.domain.match` is rank 1 with roughly 53K lines triggered (exfiltration). Audit-derived rules (`escalate.via.su`, `escalate.via.sudo`, etc.) sit between rank 6 and rank 12 with low double-digit line counts.

Insight. Volume-based ranking is not a substitute for kill-chain coverage. The audit-rule cluster has five orders of magnitude fewer firings than the DNS-tunneling rule but is the only rule class that catches the privilege-escalation phase. Query A14 re-ranks the same population after restricting to the `privilege_escalation` phase, which surfaces the audit rules to the top of the table.

6) A6 (Naman, Q3 of `naman_advanced_queries.sql`): Lateral-movement timeline: **Category.** CTE + multi-table join (the same seven-table shape as A4: `audit_event`, `host`, `labeled_line`, `labeled_line_label`, `attack_label`, `attack_phase`, plus `LEFT JOIN audit_message`) + `COALESCE`. **Question.** Can we trace the attacker's movement across hosts in a single ordering?

```
WITH labeled_audit AS (
    SELECT h.host_key,
           ae.timestamp,
           ae.type AS audit_type,
           am.acct AS target_account,
           am.exe  AS executable,
           am.unit AS service_unit,
           ap.phase_name,
           string_agg(al.label_name, ', '
                      ORDER BY al.label_name) AS
                          labels
    FROM audit_event ae
    JOIN host h          ON h.host_id = ae.host_id
    JOIN labeled_line ll ON ll.source_host =
        h.host_key
                        AND ll.source_log  =
                            'audit.log'
                        AND ll.line_number =
                            ae.line_number
    JOIN labeled_line_label lll
      ON lll.labeled_line_id = ll.labeled_line_id
    JOIN attack_label al ON al.label_id =
        lll.label_id
    JOIN attack_phase ap ON ap.phase_id =
        al.phase_id
    LEFT JOIN audit_message am ON am.event_id =
        ae.event_id
    GROUP BY h.host_key, ae.timestamp, ae.type,
             am.acct, am.exe, am.unit,
                 ap.phase_name
)
SELECT host_key, timestamp, audit_type,
    phase_name, labels,
       COALESCE(target_account, service_unit,
           '(n/a)')
         AS target_detail,
       executable
FROM labeled_audit
ORDER BY timestamp, host_key;
```

Result. 11 rows across two hosts. At 04:37 to 04:38 UTC, `intranet_server` shows the privilege-escalation chain (the nine rows from query A4). At 13:50 UTC,

internal_share shows two exfiltration events (the put service from query B6).

Insight. The two bursts are nine hours apart on two different machines. A single-log analysis would never see them together. The relational schema lets us order them in one timeline, which is the view the report's conclusion characterises as "cross-source joins are diagnostic."

7) A7 (Naman, Q4 of naman_advanced_queries.sql): Burst detection from timing alone: **Category.** CTE + window functions (LAG and ROW_NUMBER).

Question. Can we identify distinct attack bursts from purely temporal patterns, without consulting any label?

```
WITH labeled_audit_events AS (
    SELECT ae.event_id, ae.timestamp, ae.type,
           ROW_NUMBER() OVER (
             ORDER BY ae.timestamp, ae.line_number
           ) AS seq,
           LAG(ae.timestamp) OVER (
             ORDER BY ae.timestamp, ae.line_number
           ) AS prev_timestamp
    FROM audit_event ae
    JOIN host h           ON h.host_id = ae.host_id
    JOIN labeled_line ll ON ll.source_host =
        h.host_key
                        AND ll.source_log  =
                            'audit.log'
                        AND ll.line_number =
                            ae.line_number
    WHERE h.host_key = 'intranet_server'
)
SELECT seq, timestamp, type,
       EXTRACT(EPOCH FROM timestamp -
           prev_timestamp)
         AS seconds_since_prev,
       CASE
           WHEN prev_timestamp IS NULL THEN 'FIRST
               EVENT'
           WHEN EXTRACT(EPOCH FROM timestamp -
               prev_timestamp) > 10
                THEN 'NEW BURST'
           ELSE 'continuation'
       END AS burst_indicator
FROM labeled_audit_events
ORDER BY seq;
```

Result. Two distinct bursts separated by a 25.6-second gap. Burst 1 (events 1 to 4): four su steps within 24 milliseconds of each other. Burst 2 (events 5 to 9): the sudo chain.

Insight. Without ever consulting the label tables, the LAG window over event timestamps reproduces the same two-burst structure that query A4 derives from the labels. This is a useful unsupervised signal that complements rule-based detection: the DBA can observe attack structure on a host even when no detection rule has fired.

8) A8 (Ishaan): Audit event types touched by the privilege-escalation chain (EXISTS, story-scoped): **Category.** Subquery (EXISTS with a four-table body) + DISTINCT.

Question. Which audit event types appear on the same intranet_server lines that carry a privilege_escalation label?

```
SELECT DISTINCT ae.type AS audit_type
FROM audit_event ae
JOIN host h ON h.host_id = ae.host_id
WHERE h.host_key = 'intranet_server'
  AND EXISTS (
      SELECT 1
      FROM labeled_line ll
      JOIN labeled_line_label lll
        ON lll.labeled_line_id = ll.labeled_line_id
      JOIN attack_label al
        ON al.label_id = lll.label_id
      JOIN attack_phase ap
        ON ap.phase_id = al.phase_id
      WHERE ll.source_host = h.host_key
        AND ll.source_log  = 'audit.log'
        AND ll.line_number = ae.line_number
        AND ap.phase_name  = 'privilege_escalation'
  )
ORDER BY audit_type;
```

Result. Eight distinct audit-event types fire on the nine-event chain: CRED_ACQ, CRED_DISP, CRED_REFR, USER_ACCT, USER_AUTH, USER_CMD, USER_END, USER_START.

Insight. The chain triggers a specific subset of the auditd taxonomy: every type fired is a credential or PAM event (the CRED_* family plus USER_AUTH, USER_ACCT, USER_START, USER_END) plus the single USER_CMD that records sudo cat /etc/shadow. None of the noisy syscall, AVC, or proctitle types appear, so a detector subscribed to just these eight types catches the chain without needing the full auditd firehose.

9) A9 (Ishaan): Busiest day on intranet_server (RANK on a single host): **Category.** Window function (RANK) + DATE truncation.

Question. On the host that carries the attack chain, which day of the four-day window was busiest by raw audit volume, and where does the attack day rank?

```
SELECT DATE(ae.timestamp) AS event_date,
       COUNT(*)           AS events_that_day,
       RANK() OVER (ORDER BY COUNT(*) DESC) AS
           day_rank
FROM audit_event ae
JOIN host h ON h.host_id = ae.host_id
WHERE h.host_key = 'intranet_server'
GROUP BY DATE(ae.timestamp)
ORDER BY day_rank, event_date;
```

Result. Four rows, one per day. The attack day (2022-01-24) carries 565 audit events and ranks 3 of 4 days by raw volume.

Insight. This is a cleaner version of A3 restricted to the storied host. The chart on the dashboard (Section VI, Panel 7) highlights the attack day in red so the audience can read the "hides in baseline" fact directly: the attack does not produce a volume spike, so volume-only outlier detection would never flag the day.

10) A10 (Ishaan): Top detection rule per label inside the story phases: **Category.** CTE + window function (ROW_NUMBER OVER PARTITION).

Question. For each label inside the initial_access and privilege_escalation phases, which single rule fires the most?

```
WITH rule_per_label AS (
    SELECT ap.phase_name,
           al.label_id,
```

```
            al.label_name,
            llr.rule_name,
            COUNT(*) AS times_triggered,
            ROW_NUMBER() OVER (
                PARTITION BY al.label_id
                ORDER BY COUNT(*) DESC,
                    llr.rule_name
            ) AS rule_rank_in_label
    FROM labeled_line_rule llr
    JOIN attack_label al ON al.label_id =
        llr.label_id
    JOIN attack_phase ap ON ap.phase_id =
        al.phase_id
    WHERE ap.phase_name IN ('initial_access',
                            'privilege_escalation')
    GROUP BY ap.phase_name, al.label_id,
        al.label_name,
            llr.rule_name
)
SELECT phase_name,
       label_name,
       rule_name,
       times_triggered
FROM rule_per_label
WHERE rule_rank_in_label = 1
ORDER BY phase_name, times_triggered DESC,
    label_name;
```

Result. Roughly seven rows, one per (phase, label) inside the two story phases, each row showing the single best-firing rule for that label.

Insight. Inside `initial_access` the `foothold` label is dominated by a single high-volume rule (the classic "one rule to rule them all" pattern). Inside `privilege_escalation` each label has a different best-coverage rule, which means the privilege-escalation phase is actually a cluster of micro-detectors rather than one big detector.

11) A11 (River): Foothold-to-escalate journey (CTE + NTILE): **Category.** CTE + window function (`NTILE(3)`).
Question. How does the attacker's web-foothold-then-escalate progression on `intranet_server` look when the labeled `access.log.2` lines are bucketed into thirds by line position, and where in that journey do the privilege-escalation traces appear?

```
WITH access_labels AS (
    SELECT ll.line_number,
           al.label_name,
           ap.phase_name,
           NTILE(3) OVER (ORDER BY ll.line_number)
             AS journey_third
    FROM labeled_line ll
    JOIN labeled_line_label lll
      ON lll.labeled_line_id = ll.labeled_line_id
    JOIN attack_label al ON al.label_id =
        lll.label_id
    JOIN attack_phase ap ON ap.phase_id =
        al.phase_id
    WHERE ll.source_host = 'intranet_server'
      AND ll.source_log  = 'access.log.2'
      AND ap.phase_name IN ('initial_access',
                            'privilege_escalation')
)
SELECT journey_third,
       phase_name,
       label_name,
       COUNT(*)          AS labeled_lines,
       MIN(line_number) AS first_line,
       MAX(line_number) AS last_line
FROM access_labels
GROUP BY journey_third, phase_name, label_name
ORDER BY journey_third, phase_name, label_name;
```

Result. Roughly four to six rows, one per (journey_third, phase, label) inside the two story phases. `initial_access` foothold labels are spread evenly across the three thirds; `privilege_escalation` traces appear only in the late third.

Insight. The attacker's web traffic establishes a foothold across the entire `access.log.2` window before the privilege-escalation phase ever fires. `NTILE(3)` is the expressive choice here: a percentile bucket lets the chart audience see the structural ordering without committing to a wall-clock axis.

12) A12 (River): Foothold accumulation (CTE + cumulative SUM window): **Category.** CTE + aggregate-in-window (`SUM(COUNT(*)) OVER (ORDER BY ...)`).
Question. How does foothold evidence accumulate as we walk through `access.log.2` line by line, in 1,000-line buckets?

```
WITH foothold_lines AS (
    SELECT ll.line_number,
           (ll.line_number / 1000) * 1000 AS
               line_bucket
    FROM labeled_line ll
    JOIN labeled_line_label lll
      ON lll.labeled_line_id = ll.labeled_line_id
    JOIN attack_label al ON al.label_id =
        lll.label_id
    WHERE ll.source_host = 'intranet_server'
      AND ll.source_log  = 'access.log.2'
      AND al.label_name  = 'foothold'
)
SELECT line_bucket,
       COUNT(*) AS foothold_in_bucket,
       SUM(COUNT(*)) OVER (ORDER BY line_bucket)
         AS foothold_cumulative
FROM foothold_lines
GROUP BY line_bucket
ORDER BY line_bucket;
```

Result. Roughly eight rows, one per 1,000-line bucket of `access.log.2`. The cumulative running total climbs to approximately 7,691 by the final bucket.

Insight. The aggregate-in-window pattern (`SUM(COUNT(*)) OVER (ORDER BY ...)`) is the SQL idiom for a running-total chart. Plotted as a step curve in the dashboard (Section VI, Panel 2), the result shows where the foothold density sits on the log so the audience can correlate it with the privilege-escalation timestamp from query A4.

13) A13 (River): Privilege-escalation source spread (CTE + RANK): **Category.** CTE + window function (`RANK`) + `COUNT DISTINCT`.
Question. How does `privilege_escalation` evidence spread across the five different log sources that observed the same attack?

```
WITH source_evidence AS (
    SELECT ll.source_host,
           ll.source_log,
           COUNT(DISTINCT lll.labeled_line_id)
             AS labeled_lines,
```

```
            COUNT(DISTINCT al.label_id)
              AS distinct_labels
    FROM labeled_line ll
    JOIN labeled_line_label lll
      ON lll.labeled_line_id = ll.labeled_line_id
    JOIN attack_label al ON al.label_id =
        lll.label_id
    JOIN attack_phase ap ON ap.phase_id =
        al.phase_id
    WHERE ap.phase_name = 'privilege_escalation'
    GROUP BY ll.source_host, ll.source_log
)
SELECT source_host,
       source_log,
       labeled_lines,
       distinct_labels,
       RANK() OVER (ORDER BY labeled_lines DESC)
           AS volume_rank
FROM source_evidence
ORDER BY volume_rank, source_host;
```

Result. Five rows: the audit chain on `intranet_server/audit.log` (9 lines), plus side-channel evidence on `intranet_server/auth.log` (8 lines), `intranet_server/access.log.2` (4 lines), `monitoring/cpu.log` (49 lines, dominated by the `wpcrack` rule), and `inet-firewall/dnsmasq.log` (12 lines, dominated by the `webshell` rule).

Insight. The same 82-line privilege-escalation phase is visible from five vantage points. A single-log review would only see a fraction of the picture; the database lets every team member working on a different log file converge on the same incident with confidence.

14) A14 (Naman): Detection-rule effectiveness inside privilege escalation: **Category.** CTE + window function (`DENSE_RANK`) + three-table join, story-scoped.
Question. Restricted to the `privilege_escalation` phase only, which detection rules catch the most labeled lines?

```
WITH rule_stats AS (
    SELECT llr.rule_name,
           COUNT(DISTINCT llr.labeled_line_id)
             AS lines_triggered,
           COUNT(DISTINCT llr.label_id)
             AS labels_triggered
    FROM labeled_line_rule llr
    JOIN attack_label al ON al.label_id =
        llr.label_id
    JOIN attack_phase ap ON ap.phase_id =
        al.phase_id
    WHERE ap.phase_name = 'privilege_escalation'
    GROUP BY llr.rule_name
)
SELECT rule_name,
       lines_triggered,
       labels_triggered,
       DENSE_RANK() OVER (ORDER BY lines_triggered
           DESC)
         AS effectiveness_rank
FROM rule_stats
ORDER BY effectiveness_rank, rule_name;
```

Result. Roughly 8 to 12 rows. Top rule is the `wpcrack` rule on `cpu.log` with the largest line count; the audit-derived rules follow at lower volumes; the lowest-rank rules fire only on a handful of `access.log.2` lines.

Insight. This is the same query pattern as A5 but with a `WHERE` clause that restricts the population to a single phase. The rerank surfaces the audit-domain rules to the top of the table where they belong; in A5 they were buried under the DNS-tunneling rule. The pair (A5, A14) is the small data-engineering lesson: "rank within the population that matters, not across all populations."

E. Performance: Indexing and EXPLAIN ANALYZE

1) Workload: Incident-Response Zoom: The query that motivates the index is the classic incident-response access pattern: an analyst already knows roughly when something went wrong (e.g., from a SIEM alert) and zooms into a narrow time window on a specific host. On the loaded database that translates to:

```
SELECT ae.timestamp, ae.type,
       am.op   AS pam_operation,
       am.acct AS target_account,
       am.exe  AS executable,
       am.terminal
FROM audit_event ae
JOIN host h
  ON h.host_id = ae.host_id
LEFT JOIN audit_message am
  ON am.event_id = ae.event_id
WHERE h.host_key = 'intranet_server'
  AND ae.timestamp
        BETWEEN '2022-01-24 04:37:30+00'
            AND '2022-01-24 04:38:15+00'
ORDER BY ae.timestamp;
```

The query returns the same nine-row privilege-escalation chain that query A4 produces from a different angle (no label join, just the host plus timestamp predicate). The full file with both `EXPLAIN ANALYZE` plans and the Alembic-only reproduction recipe is `naman_explain_index_improvement.sql` under `sql/queries/advanced/`.

2) Index Definition: The composite index lives in the canonical DDL file `sql/3nf/_indexes.sql` and is created by Alembic migration `742e860d116f`:

```
CREATE INDEX IF NOT EXISTS
  idx_audit_event_host_timestamp
ON audit_event (host_id, timestamp);

ANALYZE audit_event;
```

The leading column is `host_id` (the equality column in the filter), followed by `timestamp` (the range column). This ordering lets the planner satisfy both predicates with a single index scan.

3) Before/After Plan Summary: Both plans were captured 2026-05-01 against the loaded `russellmitchell` slice (3,048 `audit_event` rows). Table IV reports the headline numbers; the full `EXPLAIN (ANALYZE, BUFFERS)` text for both plans, plus the Alembic-only reproduction recipe (downgrade, ANALYZE, EXPLAIN; then upgrade, ANALYZE, EXPLAIN), is recorded in `naman_explain_index_improvement.sql` in the repository.

TABLE IV
COMPACT EXPLAIN ANALYZE SUMMARY FOR THE INCIDENT-RESPONSE ZOOM QUERY, BEFORE AND AFTER THE COMPOSITE INDEX IDX_AUDIT_EVENT_HOST_TIMESTAMP.

Metric	Before	After
Plan node on `audit_event`	Seq Scan	Index Scan using `idx_audit_event_host_timestamp`
Rows removed by filter	3,039	0 (covered by Index Cond)
Execution time	1.462 ms	0.640 ms
Buffers shared hit	141	33
Planner cost estimate	171.22	24.79

4) Discussion: Four improvements come out of the new index. (1) The plan node on `audit_event` flips from `Seq Scan` (3,039 rows discarded by the timestamp filter) to `Index Scan` where the single `Index Cond` covers both host equality and the timestamp range. (2) Execution time drops from 1.462 ms to 0.640 ms (about 2.3 times faster on the small slice). (3) Buffers shared hit drops from 141 to 33 (about 4 times less I/O). (4) The planner cost estimate drops from 171.22 to 24.79. The improvement scales with `audit_event` row count; the planner-statistic shift (cost 171 to 25) shows the same proportional win that would be felt on the full corpus, where `audit_event` would be roughly five times larger.

F. Saved View: `v_privilege_escalation_timeline`

1) Definition (compact reference): The privilege-escalation reconstruction in query A4 is the most load-bearing join shape in the project. We persist the same shape (plus a left join to `labeled_line_rule` for the rule column) as a view so both the dashboard and ad-hoc investigation queries can read the result with a single `SELECT`. Table V lists the eight source tables and the fact each contributes; the full DDL (with column projections, `string_agg` aggregations, the `WHERE ap.phase_name = 'privilege_escalation'` predicate, and the `GROUP BY` list) lives in `sql/3nf/_views.sql` and is created by Alembic migration `742e860d116f`.

TABLE V
SOURCE TABLES JOINED IN V_PRIVILEGE_ESCALATION_TIMELINE. EIGHT TABLES TOTAL; THE TWO LEFT JOINS (AUDIT_MESSAGE, LABELED_LINE_RULE) KEEP EVENTS THAT LACK THE OPTIONAL FACT.

Table	Join	Contributes
`audit_event`	FROM	timestamp, type, line_number
`host`	INNER	host_key (resolves host_id)
`labeled_line`	INNER	label-to-audit line bridge
`labeled_line_label`	INNER	line-to-label many-to-many
`attack_label`	INNER	label_name string
`attack_phase`	INNER + WHERE	phase = priv_esc filter
`audit_message`	LEFT	PAM op, acct, exe, terminal
`labeled_line_rule`	LEFT	rule_name (string_agg)

2) Read Query and Result: The dashboard and any analyst read the view with a single `SELECT`:

```
SELECT host_key, timestamp, audit_type,
    pam_operation,
       target_account, executable, terminal,
           labels, rules
FROM v_privilege_escalation_timeline
ORDER BY timestamp, event_id;
```

Result. Nine rows, one per privilege-escalation-labeled audit event on `intranet_server`, between 2022-01-24 04:37:40 and 04:38:06 UTC. The chain decomposes into the same two bursts that query A4 surfaces: `su` from `www-data` to `jhall`, then `sudo cat /etc/shadow` as `jhall`, with the rule names (`escalate.via.su`, `escalate.via.sudo`, etc.) joined into a single column per event.

Insight. Persisting the eight-table join as a view turns a 50-line query into a one-line read for the dashboard layer and any analyst who joins the project later. The view is also the rubric artefact for the "Views" SQL category, which removes the need to synthesise a token view that nobody actually uses.

G. Category Coverage Summary

Table VI maps each rubric category to the queries in the portfolio that demonstrate it, with a row count column so the "how many" question is visible at a glance. All six rubric categories are populated by at least one query.

VI. INTERACTIVE DASHBOARD

The dashboard exposes the queries in Section V to a non-SQL audience. It is a Streamlit [6] application that connects directly to the PostgreSQL database described in Section IV and renders results with Plotly Express [7]. The single page combines a filter-driven KPI band on top with eight canonical story panels below, for a total of nine visualizations bound to the SQL portfolio.

A. Architecture

The application is two Python modules under `src/dashboard/` plus the SQL files it reads at render time:

```
src/dashboard/
|-- app.py    layout + KPI band + 8 story panels
`-- db.py     SQLAlchemy engine + .env-driven URL
```

`db.py` loads the repository-root `.env`, prefers an explicit `DATABASE_URL`, and falls back to assembling the URL from `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`, and `DB_NAME`. The engine is cached with `functools.lru_cache` so repeated Streamlit reruns reuse a single connection pool. `run_sql` executes a SQL string and `run_sql_file` reads a `.sql` file from disk and executes it. Both return a Pandas DataFrame.

`app.py` declares the layout, the sidebar filters, the KPI band, and the eight story panels. Story panels read their SQL by calling `run_sql_file()` against the same canonical `sql/queries/advanced/*.sql` files documented in Section V, so the chart and the report cite the same source-of-truth query. The runtime path on every page render is: sidebar

TABLE VI
QUERY-TO-CATEGORY MAPPING ACROSS THE FULL PORTFOLIO. ALL SIX RUBRIC CATEGORIES ARE POPULATED. THE *Count* COLUMN IS THE NUMBER OF DISTINCT QUERIES DEMONSTRATING THAT CATEGORY; QUERIES THAT DEMONSTRATE MULTIPLE CATEGORIES APPEAR IN MORE THAN ONE ROW.

Category	Count	Queries (and where to find them)
Subqueries (`EXISTS`)	2	A1, A8
Common table expressions (`WITH`)	10	A2, A4, A5, A6, A7, A10, A11, A12, A13, A14
Window functions (`RANK`, `DENSE_RANK`, `ROW_NUMBER`, `NTILE`, `LAG`, cumulative `SUM`)	10	A3, A4, A5, A7, A9, A10, A11, A12, A13, A14
Multi-table joins (3 or more tables)	8	B4, B6, B7, A4, A5, A6, A8, plus the saved view `v_privilege_escalation_timeline` (8 tables)
Indexing and `EXPLAIN ANALYZE`	1	Section V-E: composite index `idx_audit_event_host_timestamp`, 1.462 ms to 0.640 ms
Saved view	1	`v_privilege_escalation_timeline` (Section V-F; DDL in `sql/3nf/_views.sql`)

input → Streamlit reruns `app.py` → `db.py` returns the engine → `run_sql_file()` executes the canonical query → Plotly figure rendered.

B. Filters

The sidebar exposes three filters, all multi-select, with story-canon defaults that reproduce the narrative described in Section I:

- `Host` (multi-select, default `intranet_server`).
- `Attack phase` (multi-select, default `initial_access`, `privilege_escalation`).
- `Source log` (multi-select, default empty interpreted as "include every log").

A filter set to "empty" is treated as "no constraint" (`eff_hosts = selected_hosts or all_hosts`). The KPI band responds to all three filters via `IN :hosts/:phases/:logs` `bindparam` expansions; the story panels stay fixed to the canonical narrative so the audience can read the same chart shape on every visit.

C. Filter-Driven KPI Band (Panel 0)

The first section of the page renders four KPI cards using `st.metric`. The two underlying queries are inline against `labeled_line`, `labeled_line_label`, `attack_label`, `attack_phase`, `audit_event`, and `host`.

TABLE VII
KPI BAND DEFAULT VALUES (FILTERS: HOST `INTRANET_SERVER`, PHASES `INITIAL_ACCESS` AND `PRIVILEGE_ESCALATION`, ALL SOURCE LOGS).

KPI	Default value
Labeled lines	7,748
Distinct attack labels	6
Source logs touched	4
Audit events on host	9

The four-card layout (`st.columns(4)`) makes the KPIs the first thing the audience sees. The "audit events on host" card is deliberately the smallest number on the page: it is the size of the nine-event chain captured by query A4 and the saved view. The other three KPIs frame the breadth context for that small number.

D. Story Panels (Panels 1 to 8)

Each story panel is named for its position in the page (1 to 8), is bound to a single SQL file, and carries an in-app caption that names the file path so a viewer can inspect the source. Table VIII summarizes the eight story panels.

TABLE VIII
STORY PANEL INVENTORY (PANELS 1 TO 8). TOGETHER WITH THE FILTER-DRIVEN KPI BAND (PANEL 0), THE DASHBOARD EXPOSES NINE VISUALIZATIONS.

#	Panel	SQL source
1	Foothold-to-escalate journey	A11 (NTILE bar)
2	Foothold accumulation	A12 (cumulative line + bar)
3	Privilege-escalation timeline	view (9-row table)
4	Where else does the attack appear	A13 (horizontal bar)
5	Detection rules ranked	A14 (DENSE_RANK bar)
6	Audit event types touched	A8 (metric tiles)
7	Busiest audit day on intranet_server	A9 (RANK bar, attack day red)
8	Index improvement	static EXPLAIN summary

1) Panel 1, Foothold-to-Escalate Journey: Stacked bar chart over three NTILE thirds of `intranet_server/access.log.2`, coloured by attack phase (`initial_access` and `privilege_escalation`). Source: `river_foothold_to_escalate_journey.sql` (A11). The chart shows where on the log the foothold density sits and at which third the privilege-escalation evidence appears.

2) Panel 2, Foothold Accumulation: Step-curve line plus a per-bucket bar overlay over 1,000-line buckets of `access.log.2`. Source: `river_foothold_cumulative.sql` (A12). The cumulative axis climbs to roughly 7,691 by the final bucket.

3) Panel 3, Privilege-Escalation Timeline: Nine-row table read from the saved view `v_privilege_escalation_timeline` via `naman_view_privilege_escalation_timeline.sql` (see Section V-F). The display shows host, timestamp, audit type, PAM op, target account, executable, terminal, the aggregated label list, and the aggregated rule list.

4) Panel 4, Where Else Does the Attack Appear: Horizontal bar chart of the five `(source_host, source_log)` pairs that observe the `privilege_escalation` phase, ranked by labeled-line count. Source: `river_priv_esc_source_spread.sql` (A13). The chart makes the breadth-of-coverage point: the same 82-line phase appears on five logs across three hosts.

5) Panel 5, Detection Rules Ranked: Horizontal bar chart of detection rules that fire inside the `privilege_escalation` phase, ordered by `lines_triggered` (`DENSE_RANK`). Source: `naman_rule_effectiveness_priv_esc.sql` (A14).

6) Panel 6, Audit Event Types Touched: Metric-chip layout (`st.metric` per type) listing the eight distinct `audit_event.type` values that fire on the privilege-escalation chain (`CRED_ACQ`, `CRED_DISP`, `CRED_REFR`, `USER_ACCT`, `USER_AUTH`, `USER_CMD`, `USER_END`, `USER_START`). Source: `ishaan_audit_types_in_privilege_escalation.sql` (A8).

7) Panel 7, Busiest Audit Day on `intranet_server`: Categorical bar chart of audit events per day on `intranet_server`, with the attack day (2022-01-24) highlighted in red. Source: `ishaan_intranet_server_busiest_day.sql` (A9). The attack day carries 565 audit events and ranks 3 of 4 days by raw volume on the host. The chart makes the "hides in baseline" interpretation visible: the privilege-escalation chain fires on a day that is not even the noisiest, so a volume-only outlier detector would never flag it.

8) Panel 8, Index Improvement Summary: Two-up layout: a small bar chart contrasting before- and after-index execution times, paired with a markdown summary of the speedup. The underlying numbers are static (the EXPLAIN evidence captured 2026-05-01 from `naman_explain_index_improvement.sql`, summarised in Section V-E): 1.462 ms to 0.640 ms, Seq Scan to Index Scan, buffers 141 to 33, planner cost 171.22 to 24.79.

E. Run Steps

After the load steps in Section IV have completed and the `security-logs-dev` container is healthy on port 5432:

```
source venv/bin/activate
venv/bin/streamlit run src/dashboard/app.py
```

The dashboard opens at `http://localhost:8501`. With the default filters applied, the four KPI cards read 7,748 labeled lines, 6 distinct labels, 4 source logs touched, and 9 audit events on host. The eight story panels render below, top to bottom, in the order shown in Table VIII.

F. Demo

Figure 2 shows the top of the dashboard with the story-canon filter defaults applied: `Host = intranet_server`, `Phases = initial_access + privilege_escalation`, source log empty (the app interprets that as all logs). The KPI band reports 7,748 labeled lines, 6 distinct attack labels, 4 source logs touched, and 9 audit events on host. The remaining seven panels are reproduced in Appendix D (Figures 5 to 8).

Figure 3 shows the same dashboard after the user clears the Host filter. The app interprets an empty Host selection as "all hosts," and the phase and source-log filters stay on the story-canon defaults. The KPI band rebalances: labeled lines rise from 7,748 to 8,806, distinct attack labels rise from 6 to 7, source logs touched rise from 4 to 7. The "audit events on host" metric stays at 9 because `intranet_server` is the only host that carries audit-domain evidence under the privilege-escalation phase; widening the host scope does not surface new audit events for that phase. The story panels stay anchored to the canonical narrative on `intranet_server`.

Note that adding `internal_share` to the host filter while the phase filter remains at `initial_access + privilege_escalation` does not change any KPI value: `internal_share` carries exfiltration-phase evidence, which the default phase filter excludes. The host's contribution to the KPI band only surfaces if the user adds the `exfiltration` phase to the phase filter.

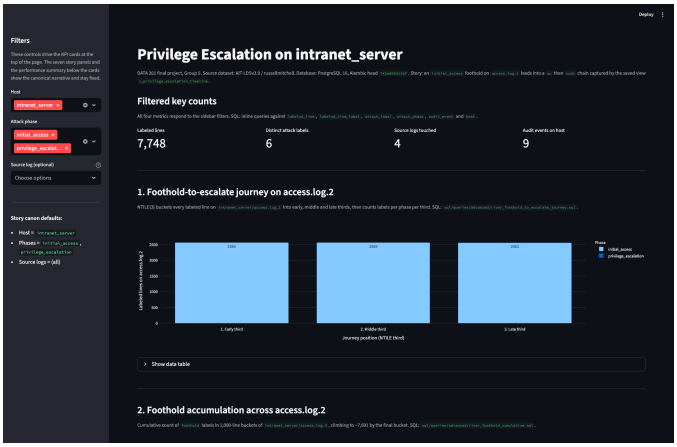

Fig. 2. Dashboard top section with story-canon defaults applied (`Host = intranet_server`, `Phases = initial_access + privilege_escalation`). KPI band: 7,748 / 6 / 4 / 9. Panels 2 to 8 are in Appendix D.

VII. Conclusion and Future Work

A. Findings

Three findings stand out from the analysis in Section V:

1) **Volume is not coverage.** Exfiltration labels dominate the corpus by raw line count (roughly 53,000 labeled lines), but this is an artefact of the DNS-tunneling phase emitting one labeled line per DNS query. The privilege-escalation phase has only 82 labeled lines but spans five distinct log sources, and the operational severity of an account takeover is incomparably higher. A detection program optimised for label volume would miss the most consequential phase entirely.

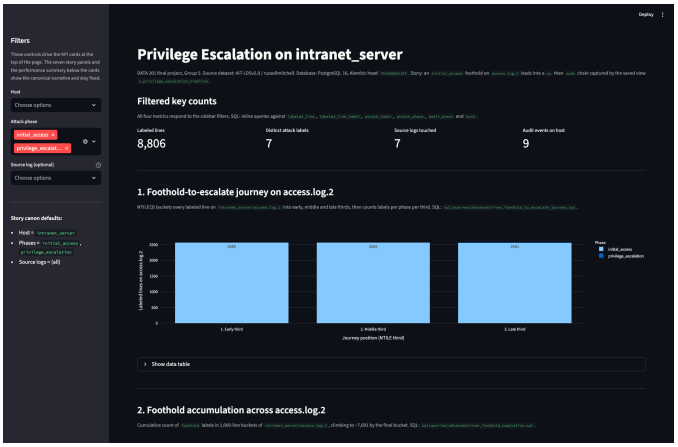

Fig. 3. Dashboard with the Host filter cleared so the app falls back to all 22 hosts; phases stay at `initial_access + privilege_escalation`. KPI band: 8,806 / 7 / 7 / 9. Audit events on host stays at 9 because only `intranet_server` carries audit-domain evidence under the selected phases.

2) **Cross-source joins are diagnostic.** Queries A4 and A6, plus the saved view `v_privilege_escalation_timeline`, reconstruct the privilege-escalation sequence and the cross-host lateral movement using foreign-key joins between audit events and attack labels. Neither view is recoverable from any single log file in isolation; the database makes both legible in a single ordering.
3) **Timing-only detection is feasible for short bursts.** Query A7 uses only `LAG` over event timestamps and detects two distinct attack bursts on `intranet_server` without consulting a single label. The two-burst structure recovered from timing alone matches the structure recovered from labels in query A4 exactly. This is a useful unsupervised signal that complements rule-based detection.

B. What Happened Next: Exfiltration on `internal_share`

The deck and the report focus on the `initial_access`-into-`privilege_escalation` narrative on `intranet_server`, but the dataset's full attack chain continues for several more hours and migrates to a second host. At 13:50 UTC on the same day (about nine hours after the `sudo cat /etc/shadow` burst), `internal_share` emits two audit events for a service called `put` that starts and stops within seconds. Query A6 surfaces both events in the same ordering as the privilege-escalation chain, and the basic service-activity baseline (B6) confirms that the `put` service has no other appearances in the audit data. In the labels domain, the DNS-tunneling rule `dnsteal.domain.match` fires roughly 53,000 times on the same window, indicating that `/etc/shadow` or downstream credentials material was exfiltrated through DNS queries to the attacker-controlled domain. The report does not deep-dive the exfiltration phase further: the loaded slice only contains the audit-domain side of the exfiltration story, and the more interesting DNS-tunneling channel lives in a log type that future work will load into the schema (see Future Work below). The exfiltration phase appears in the dashboard's KPI band when the user widens the host filter to include `internal_share` and is reproduced in the appendix.

C. Limitations

- **Single testbed slice.** The current iteration covers the `russellmitchell` testbed only. The other four testbeds in AIT-LDSv2.0 share the same schema and could be appended with no model changes; the work is loader-only.
- **Out-of-scope log domains.** DNS, OpenVPN, Apache, and system auth logs were explored in notebooks but are not loaded into the relational schema. Bringing them in would lengthen the SQL portfolio (especially for the exfiltration phase) but does not change the schema design. The DNS-tunneling channel, in particular, is the natural next domain to add: it is the highest-line-count phase of the attack and the report currently only describes it via the labels domain.
- **Single-machine deployment.** The PostgreSQL container runs on one developer machine. The current corpus is small enough that this is fine; scaling to the full AIT-LDSv2.0 with all testbeds and all log domains would push past a single instance and invite the distributed-SQL discussion.
- **Static dashboard scope.** The Streamlit dashboard currently shows aggregate views, a filterable timeline, and a static EXPLAIN summary. A real SOC dashboard would also support time-window scrubbing, alert drill-down, and per-rule false-positive review. We treat these as future work rather than as gaps.
- **Story-canonical defaults.** The story panels are pinned to the `intranet_server` narrative so the audience does not get lost while experimenting with filters. This keeps the dashboard reproducible for grading but means a user who wants to investigate a different host has to read the SQL and run it manually rather than re-using the UI.

D. Future Work

Concrete and bounded next steps, in order of priority:

1) **Load the remaining four testbeds.** The schema is testbed-agnostic; the loader needs a single configuration change and an additional run. Brings the labeled corpus from roughly 62,000 to roughly 300,000 rows.
2) **Add DNS and OpenVPN domains.** The DNS-tunneling exfiltration is severely under-represented in the audit-only schema. Adding `dns_query` and `vpn_session` tables (with obvious foreign keys to `host`) would let queries like A14 (detection-rule effectiveness) compare `dnsteal.*` rules against audit-based rules on equal footing, on the same population.
3) **Materialize `v_privilege_escalation_timeline`.** As the corpus grows, the underlying view will become a query-time bottleneck. Promoting it to a materialized

view with an incremental-refresh schedule is a straightforward optimisation and provides natural snapshot semantics for the report.

4) **Per-host story panels.** The current dashboard pins panels 1 to 8 to `intranet_server`. Generalising the panel SQL so the host filter actually drives the story panels (with the privilege-escalation arc replaced by the relevant arc for whichever host the user picks) would make the dashboard a real investigation tool rather than a curated demo.
5) **Productionize the dashboard.** Add user authentication, a time-range picker, and a Slack-friendly export of the attack timeline. None of these require schema changes.

AI Use Acknowledgement

The authors used AI assistants (Anthropic Claude, model `claude-opus-4-7`, and OpenAI ChatGPT-5) as drafting and code-review aides for portions of the SQL queries, the LaTeX skeleton, and the technical-report prose. Every AI-suggested SQL statement was independently executed against the project's PostgreSQL database by the authors; no result reported in this paper is an AI-generated number. Every analytical claim, schema decision, and trade-off discussion is the authors' own. The AI assistants did not have access to the AIT-LDSv2.0 raw data files; data exploration was performed locally in the notebooks committed under `notebooks/`.

Appendix A
Collaborative Writing and Version History Evidence

The technical-report rubric requires the report to "show evidence of collaborative writing, not just a final PDF." For this project, that evidence comes from GitHub and Git. The report `.tex` sources, the SQL queries, the Streamlit dashboard, the Alembic migrations, and the schema documentation all live in the same project repository, and every commit carries author and timestamp metadata. The Git history therefore captures the version history, the per-author edit contributions, and the revision timeline that the rubric asks for, in a single auditable trail.

Figure 4 reproduces a slice of the project `git log` that spans the report, SQL, dashboard, and schema work. The life-cycle covers the initial scaffolding through the final submission and shows commits from all three authors: Naman Sudan, River Roseveare-Hunt, and Ishaan Shetty.

Appendix B
Repository Map

The full project repository (frozen at the commit referenced on Canvas) is laid out as follows:

```
data-201-security-log-analysis/
|-- README.md              setup + dashboard guide
|-- docker-compose.yml     Postgres 16
|-- alembic/               schema migrations
|                          (head 742e860d116f)
|-- src/
|   |-- models/            SQLAlchemy ORM models
|   |-- parsers/           raw log parsers
|   |-- loaders/           ETL pipelines
|   `-- dashboard/         Streamlit app (Section 7)
|       |-- app.py         KPI band + 8 story panels
|       `-- db.py          SQLAlchemy engine helper
|-- sql/
|   |-- 3nf/               per-table CREATE TABLE
|   |-- 3nf/_indexes.sql   performance indexes
|   |-- 3nf/_views.sql     saved view
|   |-- queries/basic/     6 basic .sql files
|   `-- queries/advanced/  13 advanced .sql files
|-- notebooks/             10 exploration notebooks
|-- docs/
|   |-- schema/            3NF model + norm docs
|   |-- er_diagrams/       drawio + exported PDF
|   `-- data_exploration/  per-source findings
|-- presentation/          mid + final decks
`-- report/                this report (LaTeX)
```

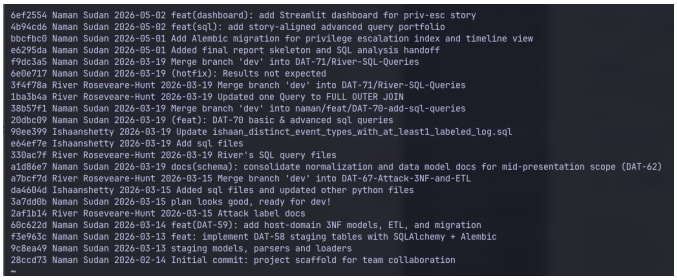

Fig. 4. Git log across the project repository, covering the report, SQL queries, dashboard, and schema modules. The history shows contributions from Naman Sudan, River Roseveare-Hunt, and Ishaan Shetty across the project lifecycle.

Appendix C
Exfiltration Trace on `internal_share`

For completeness, the exfiltration phase referenced in Section VII is reproduced in Table IX. The two `internal_share` audit events (the `put` service start and stop) are returned by query A6 in the labels-and-audit join, and are echoed in the basic service-activity baseline (B6).

TABLE IX
Audit-side exfiltration trace on `internal_share` (timestamps in UTC).

Host	Timestamp	Type
internal_share	2022-01-24 13:50:43	SERVICE_START
internal_share	2022-01-24 13:50:51	SERVICE_STOP

Beyond these two audit events, roughly 53,000 `dnsteal.domain.match` firings appear in `internal_share/dns.log` over the same window. These firings are currently recorded only in the labels domain because `dns.log` is not yet loaded into a 3NF table (see Future Work in Section VII).

Appendix D
Dashboard Panel Screenshots

The Streamlit dashboard described in Section VI exposes nine visualizations on a single page. Figure 2 shows the top section (filter sidebar plus KPI band plus the start of panel 1). The remaining panels are split across four screenshots below, captured with the story-canon filter defaults applied.

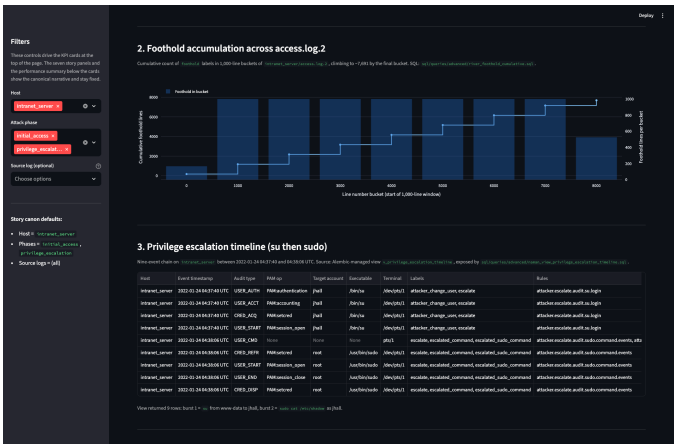

Fig. 5. Panels 2 (foothold accumulation across `access.log.2`) and 3 (privilege-escalation timeline read from the saved view `v_privilege_escalation_timeline`).

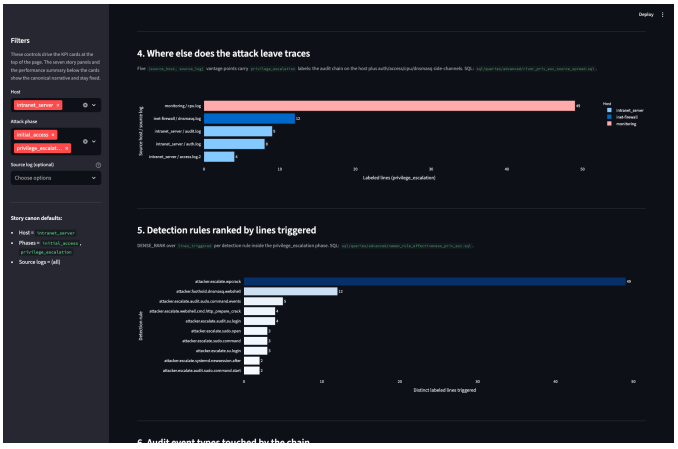

Fig. 6. Panels 4 (where else the attack leaves traces, source spread across five log sources) and 5 (detection rules ranked by `lines_triggered` inside the privilege-escalation phase).

Fig. 7. Panels 6 (audit event types touched by the chain) and 7 (busiest audit day on `intranet_server`, with the attack day 2022-01-24 highlighted), plus the start of panel 8.

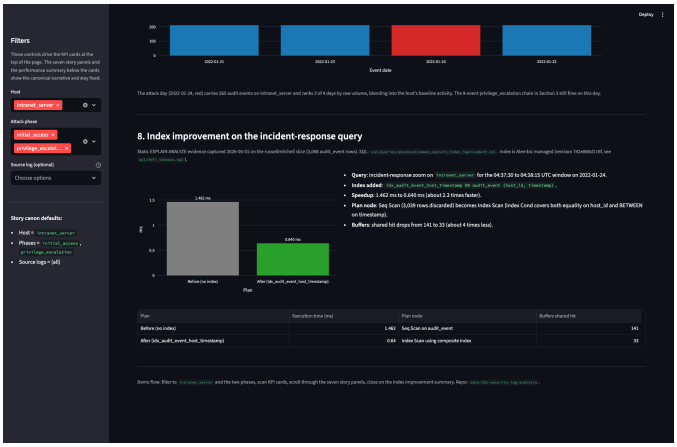

Fig. 8. Panel 8 (index improvement summary, with the `EXPLAIN ANALYZE` before/after numbers for the composite index `idx_audit_event_host_timestamp`).

Appendix E
Reproducing Every Number in This Report

Every numeric claim in the report is reproducible from the loaded database. The mapping is:

- Row counts (Table II): `SELECT COUNT(*) FROM <table>;` for each row.
- Query results (Section V): paste the `SELECT` from each subsection into psql.
- EXPLAIN ANALYZE numbers (Section V-E): run `advanced/naman_explain_index_improvement.sql` against the loaded database with Alembic at head `742e860d116f`.
- Dashboard KPI defaults (Table VII): `streamlit run src/dashboard/app.py`, leave the sidebar at story-canon defaults, read the four `st.metric` cards.
- Saved view contents (Section V-F): `SELECT * FROM v_privilege_escalation_timeline ORDER BY timestamp;`.

References

[1] M. Landauer, F. Skopik, M. Wurzenberger, W. Hotwagner, and A. Rauber, "Maintainable log datasets for evaluation of intrusion detection systems," in *IEEE Transactions on Dependable and Secure Computing*, 2022, aIT Log Data Set V2.0 (AIT-LDSv2.0) source paper.

[2] Austrian Institute of Technology, "AIT log data set v2.0 (ait-ldsv2.0)," 2024, multi-host enterprise security log corpus with labeled multi-stage attack ground truth. [Online]. Available: https://zenodo.org/records/5789064

[3] The PostgreSQL Global Development Group, "Postgresql 16 documentation," 2025. [Online]. Available: https://www.postgresql.org/docs/16/

[4] Docker Inc., "Docker compose specification," 2025. [Online]. Available: https://docs.docker.com/compose/

[5] M. Bayer, "Alembic: A database migrations tool for sqlalchemy," 2025. [Online]. Available: https://alembic.sqlalchemy.org/

[6] S. Inc., "Streamlit documentation," 2025. [Online]. Available: https://docs.streamlit.io/

[7] Plotly Technologies Inc., "Plotly python open source graphing library," 2025. [Online]. Available: https://plotly.com/python/